SectraTEK

AI FOR CYBERSECURITY AND DIGITAL FORENSICS · LAB 10

Ask the Memory



Volatility's plugin list becomes the agent's toolset: ask in plain English, and it runs the right plugin and cites the PID behind every answer.

Daniel Kwaku Ntiamoah Addai · Mobile Threat Research Lead, SectraTEK Lab · AI for Cybersecurity and Digital Forensics

YOUR INSTRUCTOR

Daniel Kwaku Ntiamoah Addai



Mobile Threat Research Lead

SectraTEK Lab

lab.sectratek.com

IN ONE LINE

I study how phones get spied on and tampered with, and build AI tools that help examiners find the evidence.

Mobile threat research
Spyware, stalkerware and device tampering.

AI for investigators
Machine learning that speeds up the search.

Teaching
AI for Cybersecurity and Digital Forensics.

TODAY'S STORY

From a raw memory image to a cited finding

1

Background

Some evidence only ever lives in RAM.

2

Before AI

Know the plugin, type the command, read the table.

3

The problem

Hundreds of plugins, thousands of rows.

4

The gap

A chatbot knows Volatility, not your image.

5

How agents help

The plugin list becomes the toolset.

6

The method

Pick, run, read, cite: one plugin per step.

7

The case

A real infected XP image with known answers.

8

Your lab

Run the analyst in the browser, then check it.

Practical runs in Google Colab: Volatility 3, a free Groq model, and a 40 MB public memory image. Nothing installed on your computer.

BACKGROUND

Some evidence only lives in RAM

A disk shows what was stored. Memory shows what was running at the moment of capture, including things that never touch the disk.

Running processes

What was executing, who started it, and with which command line.

Injected code

Malware that lives only inside another process's memory.

Network activity

Addresses a process was talking to, still held in memory.

Secrets in use

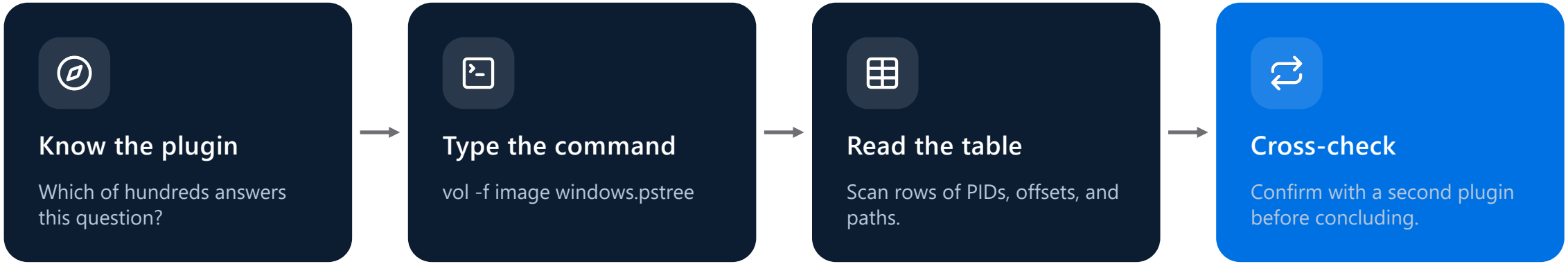
Decrypted strings, keys, and handles that vanish at shutdown.

Pull the plug and it is gone. That is why responders capture RAM first.

BEFORE AI

Volatility, one plugin at a time

Volatility is the standard open-source memory-forensics framework. Each question has its own plugin:




```
vol -f cridex.vmem windows.pstree      vol -f cridex.vmem windows.malfind --pid 1640
```

Precise and defensible, but it assumes you already know the map of plugins.

THE PROBLEM

Hundreds of plugins, thousands of rows

Even a small 512 MB image of one old PC produces a lot to read:

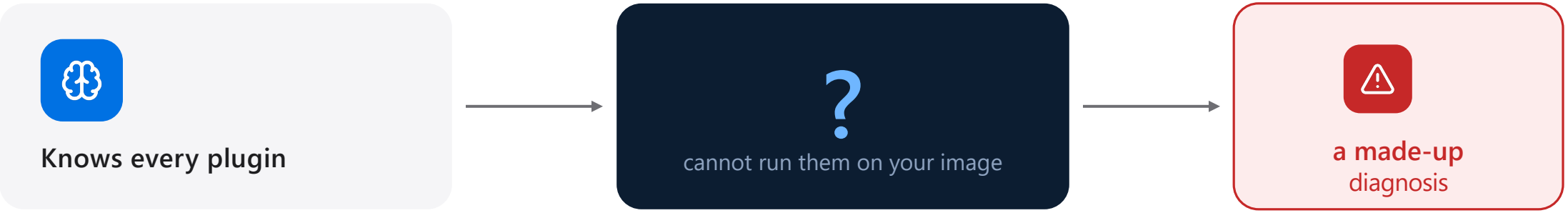
17	738	1,174	4,451
processes	loaded DLLs	file objects	open handles
`pslist`	`dlllist`	`filescan`	`handles`

> *"This machine is infected with Emotet, which is beaconing to its C2 server."*

Ask a chatbot directly and it answers anyway: fluent, specific, and invented. It never ran a single plugin.

THE GAP

It knows Volatility, but has no hands



GIVE THE MODEL HANDS

The fix is not a bigger model. Let it run the real plugins on the real image, read what comes back, and answer only from that. A model that can act on the evidence is an agent.

HOW AGENTS HELP

The plugin list becomes the toolset

Volatility already has the list of things an examiner can do. We hand that list to the model as its tools.

VOLATILITY PLUGINS

```
windows.pslist
windows.pstree
windows.malfind
windows.dlllist
windows.vadregexscan
...
```



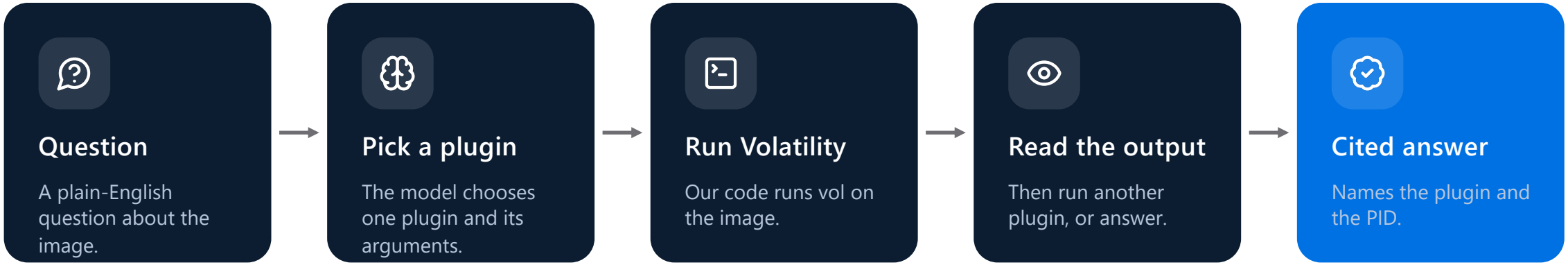
THE AGENT'S TOOLS

```
pslist()
pstree()
malfind(pid)
dlllist(pid)
search_memory(kind)
record_finding(...)
```

The model chooses the plugin and its arguments. Our code runs Volatility. Nothing else is allowed.

THE LOOP

Pick, run, read, cite



ONE PLUGIN PER STEP, AND THE REASONING IS VISIBLE

Every step shows the model's one-line thought, the plugin it ran, and what came back. The analyst can follow the trail, and so can a reviewer.

THE METHOD

A locked toolset of read-only plugins

Processes	`pslist, pstree, psscan, psxview, cmdline`
Code and modules	`malfind, dlllist, ldrmodules`
Objects	`handles, mutantscan, filescan, svcscan, hivelist, getsids, envars`
Network	`netscan (Vista+), search_memory (vadregexscan, any Windows)`
The case	`case_brief, record_finding, findings_report`

THE GUARDRAILS

Built so the agent cannot overreach

Whitelist, read-only	Only listed plugins run, without a shell. Nothing writes to the evidence.
Validated arguments	A PID must be a number. Heavy plugins require a PID or a keyword.
Checked, then cached	Each plugin is tested on this image first; only working ones become tools.
Filtered and capped	Rows are filtered by PID or keyword and capped, so nothing floods the model.
Cite or do not claim	Every answer names the plugin and PID. A malfind hit is a lead, not proof.

PLUS THE EVIDENCE CHECK, RUN BY THE CODE

Read-only tools limit what the AI can look at, not what it can say: in testing, a model invented a hidden process and a malicious.dll in this image without running one plugin. So the code checks every answer: a tool ran; every PID, IP, file, and 0x address is in tool output; no unrun tool is cited. Two tries, then it arrives marked UNVERIFIED. It catches invented details, not a misreading.

THE CASE

Case MF-2026-001: a real infected image

THE EVIDENCE

Image	cridex.vmem (public sample)
System	Windows XP SP3, 32-bit
Captured	2012-07-22, 02:45 UTC
Size	512 MB (40 MB download)
Integrity	MD5 verified on download

THE QUESTIONS

1. What system is this, and when was memory captured?
2. Which processes ran, and who launched whom?
3. Is anything unusual for this system?
4. Is there injected code? In which processes?
5. Any network indicators? In which process?
6. What can memory NOT tell us?

WORKED EXAMPLE

Three plugins to a bounded finding

1	`pstree`	reader_sl.exe (1640) was launched by explorer.exe (1484), whose parent is gone.
2	`malfind`	A PE file (MZ header) in writable, executable memory in 1484 and 1640.
3	`search_memory`	41.168.5.140:8080 and two more C2-style addresses inside explorer.exe (1484).

FINDING (CITED)

Injected code in explorer.exe and reader_sl.exe, holding C2-style addresses. malfind + vadregexscan, PIDs 1484 / 1640.

NOT A FINDING

malfind also flags csrss.exe and winlogon.exe: no PE header, typical of benign system memory.

THE HONEST LIMITS

What memory cannot tell you

No XP network plugin

Volatility 3's netscan needs Vista or later. On XP, search_memory finds the addresses but cannot prove a connection succeeded.

Naming the malware

Memory shows injection and addresses. Calling it Cridex needs outside threat intelligence, so the agent must say so.

What came before

Who sent the PDF, or what was stolen, is not in RAM. Collect the disk and network logs next.

THEN CHECK THE AI AGAINST THE ANSWER KEY

The evidence check proves the PIDs and addresses came from real plugin output, not that the agent read them correctly. So compare every claim with the case's ground-truth answer key, and note any marked UNVERIFIED.

YOUR LAB

Work the case in the browser

1	**Open in Colab**	memory_agent_jupyter.ipynb (or the chat UI notebook). Each step gives its Purpose, Input, How it works, and Output.
2	**Set up, get the image**	Parts 1-2: install Volatility 3, then download the Cridex sample and verify its hash.
3	**Run Volatility by hand**	Part 3: windows.info and windows.pstree yourself first, the old way.
4	**Wire up Sectra, work the case**	Parts 4-7: test the plugins (16 of 17 work), paste a free Groq key, ask the case questions, record findings.
5	**Check the answer key**	Parts 8-9: compare every claim; the evidence check cannot catch a misreading. Then ask your own questions.

SectraTEK

Ask the memory. Cite the plugin.

The agent does the typing; the examiner keeps the judgment. Every answer names the Volatility plugin and the PID it stands on, so it can be checked.

